

CUESTIONARIO 3: Making decisions

1. Which of the following data types can be used in a switch expression? (Choose all that apply.)

- A. enum
- B. int
- C. Byte
- D. long
- E. String
- F. char
- G. var
- H. double

Mi justificación
Las variables long y double son números con decimales, así que hay problemas con evaluar cuántos casos hay para un switch ya que son prácticamente infinitos. Por lo tanto son los únicos que no aplican. Var es válido solo si la variable declarada con var fue de los otros tipos.

1. A, B, C, E, F, G. Una expresión switch admite únicamente los tipos primitivos int, byte, short y char, junto con sus clases envolventes correspondientes Integer, Byte, Short y Character, respectivamente. Por eso las opciones B, C y F son correctas y se descartan D y H. También admite enum y String, por lo que A y E son correctas. Finalmente, switch admite var si el tipo puede resolverse como uno de los tipos de datos compatibles con switch, por lo que G también es correcta.

2. What is the output of the following code snippet? (Choose all that apply.)

```
3: int temperature = 4;  
4: long humidity = -temperature + temperature * 3;  
5: if (temperature>=4)  
6: if (humidity < 6) System.out.println("Too Low");  
7: else System.out.println("Just Right");  
8: else System.out.println("Too High");
```

- A. Too Low
- B. Just Right**
- C. Too High
- D. A NullPointerException is thrown at runtime.
- E. The code will not compile because of line 7.
- F. The code will not compile because of line 8.

Mi justificación
La línea 5 da true así que entra a la línea 6, como humidity vale 8, se ejecuta el else de la línea 7, el cual imprime "Just Right"

2. B. El código compila y se ejecuta sin problemas, por lo que D, E y F son incorrectas. Aunque dos else consecutivos en las líneas 7 y 8 pueden parecer extraños, cada uno está asociado con un if diferente: el de la línea 5 y el de la línea 6, respectivamente. El valor de humidity en la línea 4 es igual a $-4 + 12$, es decir, 8. El primer if de la línea 5 se evalúa como verdadero, por lo que se ejecuta la línea 6, que se evalúa como falsa. Esto provoca que se ejecute el else de la línea 7, imprimiendo Just Right, por lo que B es la respuesta correcta.

3. Which of the following data types are permitted on the right side of a for-each expression? (Choose all that apply.)

- A. `Double[][]`
- B. `Object`
- C. `Map`
- D. `List`
- E. `String`
- F. `char[]`
- G. `Exception`
- H. `Set`

Mi justificación
Del lado derecho de un bucle for-each debe haber un elemento contenedor iterable, con elementos que se puedan recorrer, entonces los que no cumplen son Map, Object, String y Exception.

3. A, D, F, H. Un bucle for-each admite arrays, por lo que A y F son correctas. Para `Double[][]`, cada elemento del for-each sería de tipo `Double[]`. Un for-each también admite clases que implementan `java.lang.Iterable`. Aunque esto incluye muchas clases del Collections Framework, no todas implementan `java.lang.Iterable`. Por esta razón, C es incorrecta y D y H son correctas. B, E y G son incorrectas porque no implementan `java.lang.Iterable`. Aunque `String` es una lista ordenada de caracteres, la clase no implementa la interfaz necesaria para un for-each.

4. What is the output of calling `printReptile(6)`?

```
void printReptile(int category) {  
    var type = switch(category) {  
        case 1,2 -> "Snake";  
        case 3,4 -> "Lizard";  
        case 5,6 -> "Turtle";  
        case 7,8 -> "Alligator";  
    };  
    System.out.print(type);  
}
```

- A. Snake
- B. Lizard
- C. Turtle
- D. Alligator
- E. TurtleAlligator
- F. None of the above

Mi justificación
Bastante directo, el switch entra en case 5,6 así que valdrá "Turtle". No hay error de que se ejecute case 7,8 por falta de break ya que estamos usando notación de flechas, el cual no requiere break para cada case.

4. F. El código no compila porque una expresión switch requiere que se contemplen todos los posibles valores, por lo que F es correcta. Si se añadiera un default válido, el código compilaría e imprimiría Turtle en tiempo de ejecución. A diferencia de los switch tradicionales, las expresiones switch ejecutan exactamente una rama y no utilizan break entre los case.

5. What is the output of the following code snippet?

```
List<Integer> myFavoriteNumbers = new ArrayList<>();
myFavoriteNumbers.add(10);
myFavoriteNumbers.add(14);
for (var a : myFavoriteNumbers) {
    System.out.print(a + ", ");
    break;
}

for (int b : myFavoriteNumbers) {
    continue;
    System.out.print(b + ", ");
}

for (Object c : myFavoriteNumbers)
    System.out.print(c + ", ");
```

- A. It compiles and runs without issue but does not produce any output.
- B. 10, 14,
- C. 10, 10, 14,
- D. 10, 10, 14, 10, 14,
- E. Exactly one line of code does not compile.
- F. Exactly two lines of code do not compile.
- G. Three or more lines of code do not compile.
- H. The code contains an infinite loop and does not terminate.

Mi justificación
La línea de continue del segundo bucle hace que el print continuo no se ejecute nunca ya que continue terminará la iteración para ir a la siguiente. Los demás no tienen errores

5. E. El segundo bucle for-each contiene un continue seguido de una instrucción print(). Como el continue no es condicional y siempre forma parte del cuerpo del for-each, la instrucción print() nunca puede alcanzarse. Por eso el print() no compila. Como este es el único error de compilación, E es correcta. Las demás líneas compilan sin problemas.

6. Which statements about decision structures are true? (Choose all that apply.)
- A. A for-each loop can be executed on any Collections Framework object.
 - B. The body of a while loop is guaranteed to be executed at least once.
 - C. The conditional expression of a for loop is evaluated before the first execution of the loop body.
 - D. A switch expression that takes a String and assigns the result to a variable requires a default branch.
 - E. The body of a do/while loop is guaranteed to be executed at least once.
 - F. An if statement can have multiple corresponding else statements.

Mi justificación
✓ A es true ya que los objetos de Collections son iterables ✗ B es false ya que esto solo pasa con do-while ✓ C es true ya que la condición se evalúa primero en un for antes de entrar al código del body ✗ D es false, no es necesario un default ✓ E es true por definición de do-while ✗ F es false, lo que si puede tener varios es else if

6. **C, D, E.** Un for-each puede ejecutarse sobre cualquier objeto de Collections que implemente `java.lang.Iterable`, como `List` o `Set`, pero no sobre todas las clases de Collections, como `Map`, por lo que A es incorrecta. El cuerpo de un do/while se ejecuta una o más veces, mientras que el cuerpo de un while puede ejecutarse cero o más veces, por lo que E es correcta y B incorrecta. La expresión condicional de un for se evalúa al principio de la ejecución, por lo que el for puede ejecutarse cero o más veces; C es correcta. Una expresión switch que recibe un String requiere una rama default si el resultado se asigna a una variable, por lo que D es correcta. Finalmente, cada if puede tener como máximo un else asociado, por lo que F es incorrecta.

7. Assuming weather is a well-formed nonempty array, which code snippet, when inserted independently into the blank in the following code, prints all of the elements of weather? (Choose all that apply.)

```
private void print(int[] weather) {  
    for(_____) {  
        System.out.println(weather[i]);  
    }  
}
```

- A. `int i=weather.length; i>0; i--`
- B. `int i=0; i<=weather.length-1; ++i`
- C. `var w : weather`
- D. `int i=weather.length-1; i>=0; i--`
- E. `int i=0, int j=3; i<weather.length; ++i`
- F. `int i=0; ++i<10 && i<weather.length;`
- G. None of the above

Mi justificación
Las que no aplican son: A porque nunca visitará el primer elemento weather[0] C porque no tiene definido i, por lo tanto weather[i] no existe. E porque int i = 0, int j = 3 no es válido porque el int debería ser omitido al ser variables declaradas en una sola instrucción. F porque puede ser que weather tenga más de 10 elementos y haga que ++i<10 llegue a ser false y por tanto no se continúen imprimiendo los elementos restantes.

7. **B, D.** A es incorrecta porque en la primera iteración intenta acceder a

weather[weather.length] de un array no vacío, provocando un

ArrayIndexOutOfBoundsException. B es correcta e imprime los elementos en orden.

C no compila porque i no está definido en weather[i]. Para que funcionara, habría que

modificar también el cuerpo del for-each. D también es correcta y es una forma

habitual de imprimir los elementos de un array en orden inverso. E no compila y por

eso es incorrecta. Puedes declarar varios elementos en un for, pero el tipo de dato

debe escribirse una sola vez, como en for(int i=0, j=3; ...). Finalmente, F es incorrecta

porque se salta el primer elemento del array. Como la condición se comprueba antes

de ejecutar el bucle por primera vez, el primer valor de i utilizado dentro del cuerpo

será 1.

8. What is the output of calling `printType(11)`?

```
31: void printType(Object o) {  
32:     if(o instanceof Integer bat) {  
33:         System.out.print("int");  
34:     } else if(o instanceof Integer bat && bat < 10) {  
35:         System.out.print("small int");  
36:     } else if(o instanceof Long bat || bat <= 20) {  
37:         System.out.print("long");  
38:     } default {  
39:         System.out.print("unknown");  
40:     }  
41: }
```

- A. `int`
- B. `small int`
- C. `long`
- D. `unknown`
- E. Nothing is printed.
- F. The code contains one line that does not compile.
- G. The code contains two lines that do not compile.
- H. None of the above

Mi justificación

La línea 38 no compila porque el comando <code>default</code> es solo para sentencias con <code>switch</code>

8. G. Las dos primeras instrucciones de pattern matching compilan sin problemas. La variable `bat` puede volver a utilizarse siempre que ya no esté dentro de su ámbito. Sin embargo, la línea 36 no compila. Debido al *flow scoping*, si `s` no es un `Long`, entonces `bat` no está dentro del ámbito en la expresión `bat <= 20`. La línea 38 tampoco compila porque `default` no puede utilizarse como parte de un `if/else`. Por estas dos razones, G es correcta.

9. Which statements, when inserted independently into the following blank, will cause the code to print 2 at runtime? (Choose all that apply.)

```
int count = 0;
BUNNY: for(int row = 1; row <=3; row++)
    RABBIT: for(int col = 0; col <3 ; col++) {
        if((col + row) % 2 == 0)
            _____;
        count++;
    }
System.out.println(count);
```

- A. break BUNNY
- B. break RABBIT
- C. continue BUNNY
- D. continue RABBIT
- E. break
- F. continue
- G. None of the above, as the code contains a compiler error.

Mi justificación

Inicialmente debo indicar que count++ está garantizado en aparecer al menos una vez ya que con la combinación (row, col) inicial de (1, 0) la condición de $col + row \% 2 == 0$ dará false y por tanto se ejecutará count++.

La segunda vuelta será (1,1) y esta dará true así que aquí sí se ejecutará la línea que le agreguemos.

- X A no es válida porque terminaría con ambos ciclos, y no quedaríamos con count = 1
- ✓ B es válida porque con (1,0) se ejecutará count++, con (1,1) saltaremos a row = 2, con (2,0) saltaremos a row = 3, con row(3,0) se ejecutará count++ y entonces terminaremos con count=2.
- ✓ C es válida ya que haría exactamente lo mismo que la B.
- X D no es válida ya que además de las veces que se ejecutan en B y C, se ejecutaría count++ con (2,1) y (3,2), así que terminaríamos con count = 4 o más.
- ✓ E es válido ya que equivale a lo mismo que break RABBIT.
- X F no es válida porque equivale a lo mismo que continue RABBIT.
- X G no es válida porque si compila

9. B, C, E. El código contiene un bucle anidado y una expresión condicional que se ejecuta si la suma de col + row es un número par; de lo contrario, count se incrementa. Observa que E y F son equivalentes a B y D, respectivamente, porque las instrucciones sin etiqueta se aplican al bucle más interno. Al estudiar los bucles, la primera vez que la condición es verdadera ocurre en la segunda iteración del bucle interno, cuando row vale 1 y col vale 1. A es incorrecta porque esto hace que el bucle termine inmediatamente con count establecido en 1. B, C y E siguen el mismo recorrido. Primero, count aumenta a 1 en el primer bucle interno y después se sale de

él. En la siguiente iteración del bucle externo, row vale 2 y col vale 0, por lo que se sale inmediatamente del bucle interno. En la tercera iteración del bucle externo, row vale 3 y col vale 0, por lo que count aumenta a 2. En la siguiente iteración del bucle interno, la suma es par, por lo que se sale y el programa termina. Por eso B, C y E son correctas. D y F son incorrectas porque hacen que los bucles internos y externos se ejecuten varias veces, dejando count con valor 5. No es necesario seguir todas las iteraciones; basta detenerse cuando count supere 2.

10. Given the following method, how many lines contain compilation errors? (Choose all that apply.)

```
10: private DayOfWeek getWeekDay(int day, final int thursday) {
11:     int otherDay = day;
12:     int Sunday = 0;
13:     switch(otherDay) {
14:         default:
15:             case 1: continue;
16:             case thursday: return DayOfWeek.THURSDAY;
17:             case 2,10: break;

18:             case Sunday: return DayOfWeek.SUNDAY;
19:             case DayOfWeek.MONDAY: return DayOfWeek.MONDAY;
20:     }
21:     return DayOfWeek.FRIDAY;
22: }
```

- A. None, the code compiles without issue.
- B. 1
- C. 2
- D. 3**
- E. 4
- F. 5
- G. 6
- H. The code compiles but may produce an error at runtime.

Mi justificación

La línea 15 causa error porque continue no es una expresión válida en las sentencias switch. La línea 18 causa error porque para poner una variable como condición de un case, la variable debe ser final, y Sunday solo fue declarada como int Sunday, no como int final Sunday. La línea 19 causa error porque el switch está declarado como switch(int otherDay) así que los cases deben ser números enteros, y DayOfWeek.MONDAY es un String. Son 3 líneas con errores.

10. E. Este código contiene numerosos errores de compilación, por lo que A y H son incorrectas. La línea 15 no compila porque continue no puede utilizarse dentro de un switch de esta manera. La línea 16 no es una constante de compilación porque se puede pasar cualquier valor int como parámetro. Marcarla como final no cambia esto, por lo que tampoco compila. La línea 18 no compila porque Sunday no está marcado como final. Ser *effectively final* no es suficiente. Finalmente, la línea 19 no compila

porque `DayOfWeek.MONDAY` no es un valor `int`. Aunque `switch` admite valores `enum`, cada case debe tener el mismo tipo de dato que la variable `otherDay`, que es `int`. El resto de las líneas sí compila. Como exactamente cuatro líneas no compilan, E es correcta.

11. What is the output of calling `printLocation(Animal.MAMMAL)`?

```
10: class Zoo {  
11:     enum Animal {BIRD, FISH, MAMMAL}  
12:     void printLocation(Animal a) {  
13:         long type = switch(a) {  
14:             case BIRD -> 1;  
15:             case FISH -> 2;  
16:             case MAMMAL -> 3;  
17:             default -> 4;  
18:         };  
19:         System.out.print(type);  
20:     } }
```

- A. 3
- B. 4
- C. 34
- D. The code does not compile because of line 13.
- E. The code does not compile because of line 17.
- F. None of the above

Mi justificación
Ésta es muy directa. El switch ejecuta case MAMMAL y regresa 3. X B es false porque entra a case MAMMAL antes X C es false porque en notación de flecha de switch no es necesario break, así que no ejecuta el default por accidente. X D es false porque lo que no debe ser long es el parámetro de switch, pero sí lo puedes asignar a una variable long. X E no se porque daría error pero no da. X F es false porque A es true.

11. A. El código compila y se ejecuta sin problemas, imprimiendo 3 en tiempo de ejecución, por lo que A es correcta. La instrucción default de la línea 17 es opcional porque todos los valores del enum están contemplados y puede eliminarse sin cambiar el resultado.

12. What is the result of the following code snippet?

```
3: int sing = 8, squawk = 2, notes = 0;
4: while(sing > squawk) {
5:     sing--;
6:     squawk += 2;

7:     notes += sing + squawk;
8: }
9: System.out.println(notes);
```

- A. 11
- B. 13
- C. 23
- D. 33
- E. 50
- F. The code will not compile because of line 7.

Mi justificación
El ciclo es Entra a while primera vuelta, sing = 7, squak = 4, notes = 0 + 11 = 11 Entra a while segunda vuelta, sing = 6, squak = 6, notes = 11 + 12 = 23 Se imprime notes = 23.

12. C. Antes de la primera iteración, sing = 8, squawk = 2 y notes = 0. Después de la primera iteración, sing pasa a 7, squawk a 4 y notes toma la suma de los nuevos valores de sing + squawk, es decir, $7 + 4 = 11$. Después de la segunda iteración, sing pasa a 6, squawk a 6 y notes toma el valor de sí mismo más los nuevos valores de sing + squawk: $11 + 6 + 6 = 23$. En la tercera iteración, sing > squawk es falso porque $6 > 6$ es falso. El bucle termina y se muestra el valor más reciente de notes, que es 23, por lo que C es correcta.

13. What is the output of the following code snippet?

```
2: boolean keepGoing = true;
3: int result = 15, meters = 10;
4: do {
5:     meters--;
6:     if(meters==8) keepGoing = false;
7:     result -= 2;
8: } while keepGoing;
9: System.out.println(result);
```

- A. 7
- B. 9
- C. 10
- D. 11**
- E. 15
- F. The code will not compile because of line 6.
- G. The code does not compile for a different reason.

Mi justificación
El ciclo es. Entra a do primera vuelta, meters = 9, result = 15 – 2 = 13 keepGoing = true, entonces while es true y se repite el ciclo Entra a do segunda vuelta, meters = 8, keepGoing = false, result = 13 – 2 = 11 Se imprime result = 11

13. G. Este ejemplo puede parecer complicado, pero el código no compila. A la línea 8 le faltan los paréntesis obligatorios alrededor de la expresión booleana. Como el código no compila y no es debido a la línea 6, G es correcta. Si se corrigiera la línea 8 agregando los paréntesis, el bucle se ejecutaría dos veces y la salida sería 11.

14. Which statements about the following code snippet are correct? (Choose all that apply.)

```
for(var penguin : new int[2])
    System.out.println(penguin);
var ostrich = new Character[3];
for(var emu : ostrich)
    System.out.println(emu);
List<Integer> parrots = new ArrayList<Integer>();
for(var macaw : parrots)
    System.out.println(macaw);
```

- A. The data type of penguin is Integer.
- B. The data type of penguin is int.
- C. The data type of emu is undefined.
- D. The data type of emu is Character.
- E. The data type of macaw is List.
- F. The data type of macaw is Integer.
- G. None of the above, as the code does not compile.

Mi justificación
No veo problemas o trucos con el código. Las 3 estructuras de datos int[], Character[] y arrayList<Integer> son compatibles con los ciclos for each así que puede imprimir sin problemas cada elemento de los 3 tipos, y cada uno con el tipo que se declaró. Así que B, D y F son las correctas.

14. B, D, F. El código sí compila, por lo que G es incorrecta. En el primer for-each, el lado derecho tiene tipo int[], por lo que cada elemento penguin tiene tipo int; B es correcta. En el segundo for-each, ostrich tiene tipo Character[], por lo que emu tiene tipo Character; D es correcta. En el último for-each, parrots tiene tipo List<Integer>. Como el tipo genérico utilizado en List es Integer, macaw tendrá tipo Integer, por lo que F es correcta.

15. What is the result of the following code snippet?

```
final char a = 'A', e = 'E';
char grade = 'B';
switch (grade) {
    default:
    case a:
    case 'B': 'C': System.out.print("great ");
    case 'D': System.out.print("good "); break;
    case e:
    case 'F': System.out.print("not good ");
}
```

- A. great
- B. great good
- C. good
- D. not good
- E. The code does not compile because the data type of one or more case statements does not match the data type of the switch variable.
- F. None of the above

Mi justificación
La línea con case 'B': 'C': tiene una sintaxis no válida, en lugar de dos puntos entre B y C debía ser coma. Por esto, el código no compila y no imprimiría nada.

15. F. El código no compila, aunque no por la razón indicada en E. El segundo case contiene una sintaxis inválida. Cada instrucción case debe contener la palabra clave case; no puedes encadenarlas utilizando únicamente :. Por eso F es correcta. La línea podría corregirse escribiendo case 'B', 'C' o agregando la palabra case antes de 'C'. Entonces el resto del código compilaría e imprimiría great good en tiempo de ejecución.

16. Given the following array, which code snippets print the elements in reverse order from how they are declared? (Choose all that apply.)

```
char[] wolf = {'W', 'e', 'b', 'b', 'y'};
```

A.

```
int q = wolf.length;
for( ; ; ) {
    System.out.print(wolf[--q]);
    if(q==0) break;
}
```

B.

```
for(int m=wolf.length-1; m>=0; --m)
    System.out.print(wolf[m]);
```

C.

```
for(int z=0; z<wolf.length; z++)
    System.out.print(wolf[wolf.length-z]);
```

D.

```
int x = wolf.length-1;
for(int j=0; x>=0 && j==0; x--)
    System.out.print(wolf[x]);
```

E.

```
final int r = wolf.length;
for(int w = r-1; r>-1; w = r-1)
    System.out.print(wolf[w]);
```

F.

```
for(int i=wolf.length; i>0; --i)
    System.out.print(wolf[i]);
```

G. None of the above

Mi justificación

✓ A es válida porque `--q` hará que se recorran los índices del arreglo comenzando desde `length-1` hasta 0. El `break` evitará que el ciclo se ejecute infinitamente.

✓ B es válida porque implementa lo mismo que A pero en los parámetros del `for`.

✗ C no es válida porque el primer elemento sería `Wolf[5-0]` lo cual es `Wolf[5]` y ese elemento no existe ya que el máximo es `Wolf[4]`.

✓ D es válida ya que es similar a B solo que `Wolf.length-1` está antes de entrar al ciclo `for`. La presencia de `j` y de `&& j == 0` no afecta al código.

✗ E no es válida ya que `w = r-1` hace que sea un ciclo infinito ya que `int r` es de tipo final, así que `w = r-1` siempre valdrá lo mismo sin poder limitar las iteraciones.

✗ F no es válida porque el primer elemento sería `Wolf[5-0]` lo cual es `Wolf[5]` y ese elemento no existe ya que el máximo es `Wolf[4]`.

✗ G no es válida porque A, B y D son válidas

16. A, B, D. Para imprimir los elementos del array `wolf` en orden inverso, el código debe comenzar con `wolf[wolf.length-1]` y terminar con `wolf[0]`. A hace esto y es la primera correcta. B también es correcta y es una de las formas más comunes de escribir un bucle inverso. La condición de terminación suele ser `m >= 0` o `m > -1`, y

ambas son correctas. C y F provocan un `ArrayIndexOutOfBoundsException` en tiempo de ejecución porque ambas intentan acceder primero a `wolf[wolf.length]`, un índice que está fuera del rango del array de índice basado en cero. C funcionaría si se cambiara a `wolf[wolf.length-z-1]`. D también es correcta, ya que `j` es innecesaria y puede ignorarse en este ejemplo. E es incorrecta y produce un bucle infinito, porque `w` se establece repetidamente en `r-1`, en este caso 4, en cada iteración. Como la actualización no tiene efecto después de la primera iteración, la condición nunca se cumple y el bucle nunca termina.

17. What distinct numbers are printed when the following method is executed? (Choose all that apply.)

```
private void countAttendees() {  
    int participants = 4, animals = 2, performers = -1;  
    while((participants = participants+1) < 10) {}  
    do {} while (animals++ <= 1);  
    for( ; performers<2; performers+=2) {}  
  
    System.out.println(participants);  
    System.out.println(animals);  
    System.out.println(performers);  
}
```

- A. 6
- B. 3**
- C. 4
- D. 5
- E. 10**
- F. 9
- G. The code does not compile.
- H. None of the above

Mi justificación
El primer ciclo while se ejecutará hasta que participants < 10 sea false, y esto se cumple cuando participants = 9 y entra al ciclo otra vez haciendo que participants++ o bien participants = 10. Siguiendo la misma lógica, el segundo ciclo while se ejecutará hasta que animals <= 1 sea false, y esto se cumple en la primera vuelta, así que queda como animals++ o bien animals = 3. El tercer ciclo se ejecuta dos veces, performers pasa de -1 a 1, luego a 3, y de ahí rompe el ciclo porque performers < 2 es false si performers = 3.

17. **B, E.** El código compila sin problemas y muestra dos números distintos en tiempo de ejecución, por lo que G y H son incorrectas. El primer bucle se ejecuta cinco veces en total y termina cuando participants tiene valor 10. Por eso E es correcta. En el segundo bucle, animals inicialmente no es menor o igual que 1, pero como es un do/while, se ejecuta al menos una vez. De esta forma, animals toma el valor 3 y el bucle termina, por lo que B es correcta. Finalmente, el último bucle se ejecuta dos veces: performers comienza en -1, pasa a 1 al final de la primera iteración y termina con valor 3 después de la segunda, lo que rompe el bucle. Esto hace que B sea correcta por segunda vez.

18. Which statements about pattern matching and flow scoping are correct? (Choose all that apply.)
- A. Pattern matching with an `if` statement is implemented using the `instance` operator.
 - B. Pattern matching with an `if` statement is implemented using the `instanceon` operator.
 - C. Pattern matching with an `if` statement is implemented using the `instanceof` operator.
 - D. The pattern variable cannot be accessed after the `if` statement in which it is declared.
 - E. Flow scoping means a pattern variable is only accessible if the compiler can discern its type.
 - F. Pattern matching can be used to declare a variable with an `else` statement.

Mi justificación
X A y B son false porque la notación correcta es instanceof ✓ C es true porque esa es la notación correcta del pattern matching. ✓ D es true porque el if genera un scope en la que solo existen las variables dentro de este, por lo tanto las variables no pueden ser accedidas después del if. ✓ E es true porque la variable solo es definida si se cumple la condición del if. X F es false porque en ese caso necesitaríamos un elseif, no un else.

18. C, E. El pattern matching con un `if` se implementa utilizando el operador `instanceof`, por lo que C es correcta y A y B son incorrectas. D es incorrecta porque es posible acceder a una variable de patrón fuera del `if` en el que fue definida, dependiendo del *flow scoping*. E es una afirmación correcta sobre *flow scoping*. F es incorrecta. El pattern matching no permite declarar variables en los `else`, ya que los `else` no tienen una expresión booleana.

19. What is the output of the following code snippet?

```
2: double iguana = 0;
3: do {
4:     int snake = 1;
5:     System.out.print(snake++ + " ");
6:     iguana--;
7: } while (snake <= 5);
8: System.out.println(iguana);
```

- A. 1 2 3 4 -4.0
- B. 1 2 3 4 -5.0
- C. 1 2 3 4 5 -4.0
- D. 0 1 2 3 4 5 -5.0
- E. The code does not compile.
- F. The code compiles but produces an infinite loop at runtime.
- G. None of the above

Mi justificación

El código no compila ya que la condición del while no detecta el valor de snake ya que esta solo existe dentro de las llaves de do.

19. E. La variable snake se declara dentro del cuerpo de la instrucción do/while, por lo que está fuera de ámbito en la línea 7. Por esta razón, E es correcta. Si snake se hubiera declarado antes de la línea 3 con un valor de 1, entonces la salida habría sido 1 2 3 4 5 -5.0 y G habría sido la respuesta correcta.

20. Which statements, when inserted into the following blanks, allow the code to compile and run without entering an infinite loop? (Choose all that apply.)

```
4:  int height = 1;
5:  L1: while(height++ <10) {
6:      long humidity = 12;
7:      L2: do {
8:          if(humidity-- % 12 == 0) _____;
9:          int temperature = 30;
10:         L3: for( ; ; ) {
11:             temperature++;
12:             if(temperature>50) _____;
13:         }
14:     } while (humidity > 4);
15: }
```

- A. break L2 on line 8; continue L2 on line 12
- B. continue on line 8; continue on line 12
- C. break L3 on line 8; break L1 on line 12
- D. continue L2 on line 8; continue L3 on line 12
- E. continue L2 on line 8; continue L2 on line 12
- F. None of the above, as the code contains a compiler error

Mi justificación
Necesitamos líneas de código que nos permitan escapar del tercer ciclo pues si no lo hacemos, entramos en una iteración infinita. Por lo tanto, las opciones que hagan break son las correctas.

20. A, E. Lo más importante al leer este código es notar que el bucle más interno es un bucle infinito. Por lo tanto, hay que buscar soluciones que eviten completamente ese bucle o que permitan salir de él. A es correcta porque break L2 en la línea 8 hace que el segundo bucle interno termine cada vez que se entra en él, evitando completamente el bucle más interno. En B, el primer continue de la línea 8 evita el bucle más interno en la primera iteración del segundo bucle, pero no en la segunda. Entonces el bucle más interno se ejecuta y, con el continue de la línea 12, produce un bucle infinito en tiempo de ejecución. Por eso B es incorrecta. C es incorrecta porque contiene un error de compilación: la etiqueta L3 no es visible fuera de su bucle. D es incorrecta porque es equivalente a B, ya que un break o continue sin etiqueta se aplica al bucle más cercano y produce el bucle infinito. Al igual que A, continue L2 en la línea 8 permite que el bucle más interno se ejecute la segunda vez que se llama al segundo bucle. Sin embargo, el continue L2 de la línea 12 sale del bucle infinito y

devuelve el control al segundo bucle. Como el primer y segundo bucle terminan, el código termina y E es correcta.

21. A minimum of how many lines need to be corrected before the following method will compile?

```
21: void findZookeeper(Long id) {  
22:     System.out.print(switch(id) {  
23:         case 10 -> {"Jane"}  
24:         case 20 -> {yield "Lisa"}};  
25:         case 30 -> "Kelly";  
26:         case 30 -> "Sarah";  
27:         default -> "Unassigned";  
28:     });  
29: }
```

- A. Zero
- B. One
- C. Two
- D. Three
- E. Four
- F. Five

Mi justificación
La línea 22 tiene error porque en un switch no podemos usar valores long como parámetro. La línea 24 tiene error porque tiene un ; de más fuera de la llave. Las líneas 25 y 26 tienen un error ya que no puedes repetir un case. En este caso case 30.

21. E. La línea 22 no compila porque Long no es un tipo compatible con una instrucción o expresión switch. La línea 23 no compila porque falta un punto y coma después de "Jane" y una instrucción yield. La línea 24 no compila porque contiene un punto y coma adicional al final. Finalmente, las líneas 25 y 26 no compilan porque utilizan el mismo valor de case. Al menos uno de ellos tendría que cambiarse para que el código compile. Como deben corregirse cuatro líneas, E es correcta.

22. What is the output of the following code snippet? (Choose all that apply.)

```
2: var tailFeathers = 3;
3: final var one = 1;
4: switch (tailFeathers) {
5:     case one: System.out.print(3 + " ");
6:     default: case 3: System.out.print(5 + " ");
7: }
8: while (tailFeathers > 1) {
9:     System.out.print(--tailFeathers + " "); }
```

- A. 3
- B. 5 1
- C. 5 2
- D. 3 5 1
- E. 5 2 1
- F. The code will not compile because of lines 3–5.
- G. The code will not compile because of line 6.

Mi justificación
La línea 6 causa error de compilación ya que la notación de default: case 3: está mal, no puedes poner dos casos así seguidos.

22. E. El código compila sin problemas, por lo que F y G son incorrectas. Recuerda que var se admite tanto en switch como en while, siempre que el compilador pueda determinar que el tipo es compatible con estas instrucciones. Además, la variable one puede utilizarse en un case porque es una variable local final, por lo que es una constante de compilación. El valor de tailFeathers es 3, que coincide con el segundo case, haciendo que la primera salida sea 5. El bucle while se ejecuta dos veces, y el operador de predecremento (--) modifica tailFeathers de 3 a 2 y luego a 1 en la segunda iteración. Por eso la salida final es 5 2 1, haciendo que E sea correcta.

23. What is the output of the following code snippet?

```
15: int penguin = 50, turtle = 75;  
16: boolean older = penguin >= turtle;  
17: if (older = true) System.out.println("Success");  
18: else System.out.println("Failure");  
19: else if(penguin != 50) System.out.println("Other");
```

- A. Success
- B. Failure
- C. Other
- D. The code will not compile because of line 17.
- E. The code compiles but throws an exception at runtime.
- F. None of the above

Mi justificación
El orden de uso de if, else y else if es incorrecto, el orden correcto es if, else if, else. Por esto no compilará.

23. F. La línea 19 comienza con una instrucción else, pero no existe un if anterior al que corresponda. Por eso la línea 19 no compila y F es correcta. Si se eliminara la palabra else de la línea 19, el fragmento imprimiría Success.

24. Which of the following are possible data types for friends that would allow the code to compile? (Choose all that apply.)

```
for(var friend in friends) {  
    System.out.println(friend);  
}
```

- A. Set
- B. Map
- C. String
- D. int[]
- E. Collection
- F. StringBuilder
- G. None of the above

Mi justificación
Los for-each utilizan : en Java, no la palabra in. Entonces no compilará con ninguno.

24. G. La instrucción no es un for-each válido ni un for tradicional, ya que utiliza la palabra clave inexistente in. Por eso el código no compila y G es correcta. Si in se cambiara por dos puntos (:), Set, int[] y Collection serían correctos.

25. What is the output of the following code snippet?

```
6: String instrument = "violin";
7: final String CELLO = "cello";
8: String viola = "viola";
9: int p = -1;
10: switch(instrument) {
11:     case "bass" : break;
12:     case CELLO : p++;
13:     default: p++;
14:     case "VIOLIN": p++;
15:     case "viola" : ++p; break;
16: }
17: System.out.print(p);
```

- A. -1
- B. 0
- C. 1
- D. 2**
- E. 3
- F. The code does not compile.

Mi justificación
El switch entra al case default, se ejecuta p++, al no usar notación de flecha, necesita un break, el cual no tiene, por lo tanto ocurrirá un fall through y se ejecutará otro p++ y ++p. El resultado final es p = 2.

25. D. El código compila sin problemas, por lo que F es incorrecta. La variable viola creada en la línea 8 nunca se utiliza y puede ignorarse. Si se hubiera utilizado como valor del case en la línea 15, habría provocado un error de compilación porque no está marcada como final. Como "violin" y "VIOLIN" no coinciden exactamente, se ejecuta la rama default del switch. Esta ejecución incrementa p tres veces, dejando su valor final en 2, por lo que D es correcta.

26. What is the output of the following code snippet? (Choose all that apply.)

```
9:  int w = 0, r = 1;
10:  String name = "";
11:  while(w < 2) {
12:      name += "A";
13:      do {
14:          name += "B";
15:          if(name.length()>0) name += "C";
16:          else break;
17:      } while (r <=1);
18:      r++; w++; }
19:  System.out.println(name);
```

- A. ABC
- B. ABCABC
- C. ABCABCABC
- D. Line 15 contains a compilation error.
- E. Line 18 contains a compilation error.
- F. The code compiles but never terminates at runtime.
- G. The code compiles but throws a `NullPointerException` at runtime.

Mi justificación
El do while no terminará nunca ya que el <code>name.length() > 0</code> siempre será true, así que no hay forma de ejecutar break, y porque dentro del do no hay nada que altere el valor de r, entonces siempre valdrá <code>r = 1</code> .y <code>while (r <= 1)</code> siempre dará true.

26. F. El fragmento de código no contiene errores de compilación, por lo que D y E son incorrectas. Sin embargo, existe un problema. Aunque puede parecer complicado, la clave está en observar que la variable r se actualiza fuera del bucle do/while. Esto está permitido desde el punto de vista de compilación porque fue definida antes del bucle, pero significa que el bucle interno nunca rompe la condición de terminación `r <= 1`. En tiempo de ejecución, esto produce un bucle infinito la primera vez que se entra en el bucle interno, por lo que F es correcta.

27. What is printed by the following code snippet?

```
23: byte amphibian = 1;
24: String name = "Frog";
25: String color = switch(amphibian) {
26:     case 1 -> { yield "Red"; }
27:     case 2 -> { if(name.equals("Frog")) yield "Green"; }
28:     case 3 -> { yield "Purple"; }
29:     default -> throw new RuntimeException();
30: };
31: System.out.print(color);
```

- A. Red
- B. Green
- C. Purple
- D. RedPurple
- E. An exception is thrown at runtime.
- F. The code does not compile.

Mi justificación
Esto es muy directo. El swtich entra en case 1. Obtiene color = "Red" y sale del switch. No necesita break porque es notación de flecha.

27. F. La línea 27 no compila porque el bloque del case no produce un valor mediante yield si name no es igual a Frog. Por eso F es correcta. Cada posible camino dentro de un bloque case debe producir un valor si se espera que la expresión switch devuelva un valor.

28. What is the output of calling `getFish("goldie")`?

```
40: void getFish(Object fish) {  
41:     if (!(fish instanceof String guppy))  
42:         System.out.print("Eat!");  
43:     else if (!(fish instanceof String guppy)) {  
44:         throw new RuntimeException();  
45:     }  
46:     System.out.print("Swim!");  
47: }
```

- A. Eat!
- B. Swim!
- C. Eat! followed by an exception.
- D. Eat!Swim!
- E. An exception is printed.
- F. None of the above

Mi justificación
Aunque el parámetro sea Object, fish es un String que hereda cosas de objetos, así que es un parámetro válido. La condición <code>fish instanceof String</code> es true, pero tiene signo de admiración que cambia el valor booleano, así que tanto el if como el else if no operan porque la condición da false. Entonces se ejecuta solo Swim al final del código.

28. F. Basándonos en el *flow scoping*, guppy está dentro del ámbito después de las líneas 41–42 si el tipo no es String. En este caso, la línea 43 declara una variable guppy que es un duplicado de la variable local previamente definida en la línea 41. Por eso el código no compila y F es correcta. Si se utilizara un nombre de variable diferente en la línea 43, el código compilaría e imprimiría Swim! con la entrada indicada.

29. What is the result of the following code?

```
1: public class PrintIntegers {  
2:     public static void main(String[] args) {  
3:         int y = -2;  
4:         do System.out.print(++y + " ");  
5:         while(y <= 5);  
6:     } }
```

- A. -2 -1 0 1 2 3 4 5
- B. -2 -1 0 1 2 3 4
- C. -1 0 1 2 3 4 5 6
- D. -1 0 1 2 3 4 5
- E. The code will not compile because of line 5.
- F. The code contains an infinite loop and does not terminate.

Mi justificación

La operación ++y se ejecuta antes del print, así que el primer valor impreso es y = -1, cuando y valga 6, el while dará false. Entonces se imprimirán los valores del -1 al 6.
--

29. C. Como se utilizó el operador de preincremento, el primer valor que se mostrará es -1, por lo que A y B son incorrectas. En la penúltima iteración del bucle, y se incrementará a 5 y el bucle mostrará 5. El bucle continuará porque $5 \leq 5$ es verdadero y, en la última iteración, se mostrará 6. Al terminar esta última iteración, la expresión booleana $6 \leq 5$ será falsa y el bucle terminará. Como 6 fue el último valor mostrado por el bucle, C es la respuesta correcta.

Resultado final: 18/29

Calificación = 6.2